



UTEC Posgrado



UTEC Posgrado

Una escuela para
Early Adopters

CURSO

INGENIERÍA DE DATOS

Mg. Angel Tintaya
Senior Data Engineer



A hand is shown holding a glowing, translucent sphere. The sphere is covered in a network of white lines and dots, resembling a molecular structure or a data network. The background is dark blue, and the sphere itself is bright blue with a white glow. The text is centered over the sphere.

DATAOPS Y AUTOMATIZACIÓN DE PROCESOS DE DATOS



INTRODUCCIÓN A DATAOPS

CI/CD PARA INGENIERÍA DE DATOS

Son las 2:00 a.m. y un pipeline crítico acaba de fallar en producción....



¿Qué crees que es más probable que haya ocurrido?



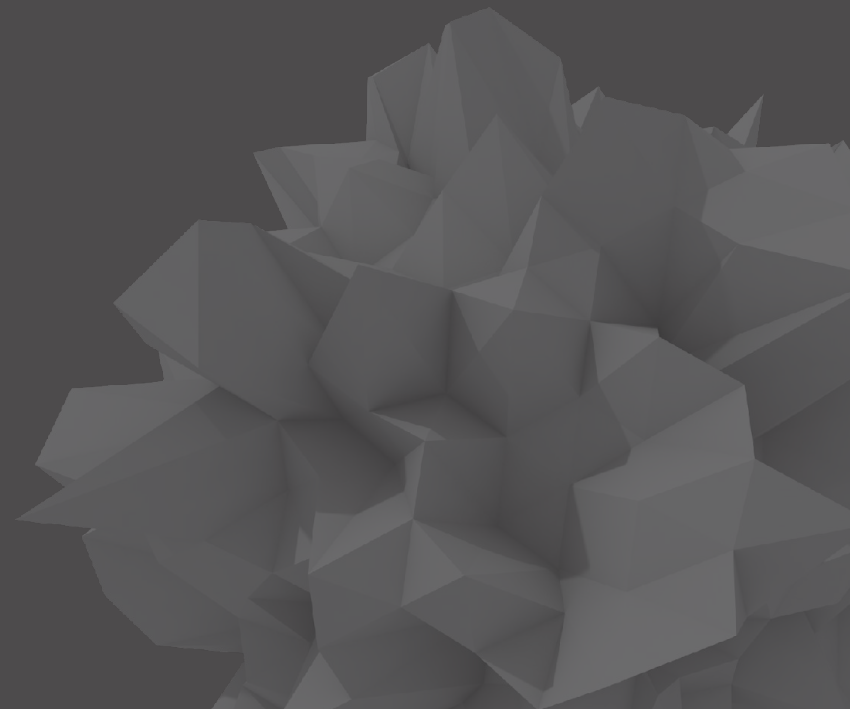
Qué aprenderemos hoy

- Qué es DataOps y por qué surgió
- Cómo DataOps extiende las ideas de DevOps
- Cómo funciona CI/CD en ingeniería de datos
- Qué prácticas permiten pipelines confiables
- Cómo automatizar validaciones y despliegues



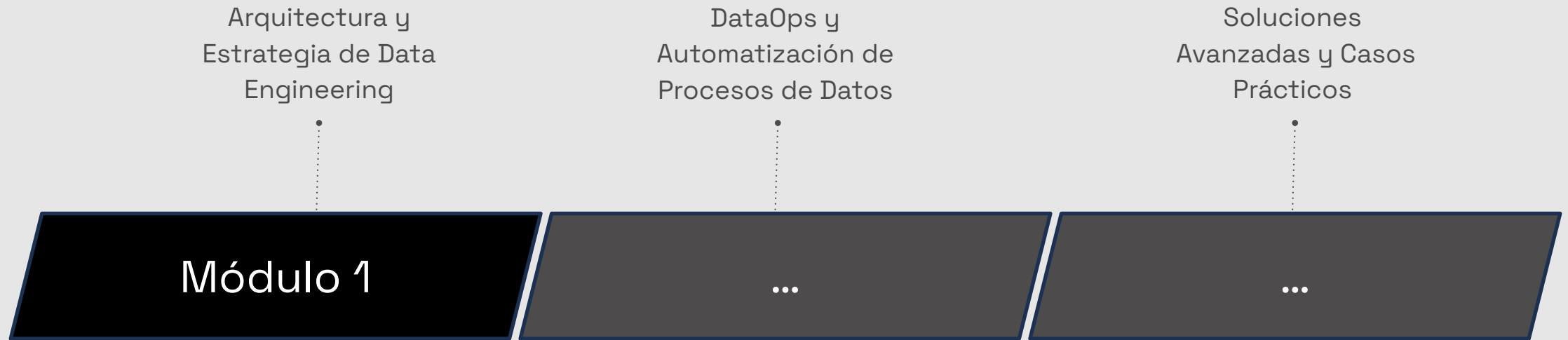
AGENDA DE HOY

- The road so far
- ¿Por qué DataOps?
- CI/CD para Ingeniería de Datos
- Buenas prácticas DataOps
- Caso práctico: GitHub Actions
- Herramientas y ecosistema





The road so far



- Fundamentos de Data Engineering
- Diferencias entre Data Engineering, Data Discovery y Data Science
- Arquitecturas modernas de datos: Data Warehouses, Data Lakes y Lakehouse
- Data Mesh y Data Fabric: enfoques modernos para la gestión de datos
- Patrones de ingestión de datos: Batch, Streaming y Lambda/Kappa Architectures





- Orquestación de flujos de datos.
- **Introducción a DataOps: CI/CD para ingeniería de datos.**
- ETL (Extract, Transform, Load) y ELT: conceptos y diferencias.
- Implementación de flujos de integración de datos con herramientas modernas.



UTEC VENTURES

¿POR QUÉ
DATAOPS?

PANEL UV

¿Cuáles de estos problemas han enfrentado en sus proyectos con datos?

- Esquemas cambiantes que rompen el flujo.
- Errores por validaciones manuales.
- Dificultad para reproducir pipelines.
- Falta de colaboración entre DS e ingenieros
- Retrasos por pasos manuales de despliegue



¿Qué es DataOps?

DataOps es una práctica colaborativa de ingeniería de datos enfocada en:

- Automatizar el ciclo de vida de los datos.
- Asegurar la calidad, trazabilidad y gobernanza.
- Facilitar la entrega continua de productos de datos confiables.



¿Qué es DataOps?

Inspirado en DevOps, pero adaptado a:

- Pipelines de datos en lugar de solo código.
- Validaciones de datos en lugar de solo tests de software.
- Entregas iterativas de insights y modelos.



A photograph of three people—two men and one woman—standing on a wide, modern concrete staircase. The man on the left is wearing a blue blazer over a light blue shirt and dark trousers. The woman in the center is wearing a light grey blazer over a mustard yellow top and dark trousers. The man on the right is wearing a light blue blazer over a light blue shirt and dark trousers. They are all smiling and looking towards the camera. The background shows the architectural details of the staircase and building, with a dark overlay and the text 'DataOps y DevOps' centered in white.

DataOps y DevOps

DevOps vs DataOps

Aspecto	DevOps	DataOps
Unidad principal	Software (apps, APIs)	Pipelines de datos + datos
Validaciones	Tests de código, cobertura	Validaciones de datos, calidad, esquema
Objetivo final	Entrega continua de software confiable	Entrega continua de productos de datos confiables
Observabilidad	Logs, métricas	Logs + monitoreo de calidad de datos
Stakeholders Clave	Devs, Ops	Devs, Ing. De Datos, Científico de Datos, Negocio



CI/CD para Ingeniería de Datos

CI/CD en DataOps: ¿Qué significa?

CI (Integración Continua)

- Automatizar la validación del código de pipelines y la calidad de datos.

CD (Despliegue Continuo)

- Automatizar el despliegue de flujos de datos a ambientes controlados (dev → qa → prod).

CI/CD en DataOps: ¿Qué significa?

Ejemplos en ingeniería de datos:

- Validar la sintaxis de un DAG en Airflow automáticamente.
- Verificar que una transformación no rompa el esquema esperado.
- Desplegar un flujo actualizado cuando se hace merge a main.



Un desarrollador modifica un DAG y rompe una transformación crítica ...

¿Cuándo debería detectarse el problema?



- a) En producción
- b) En QA
- c) Durante CI
- d) Durante CD
- e) Cuando reporte el negocio

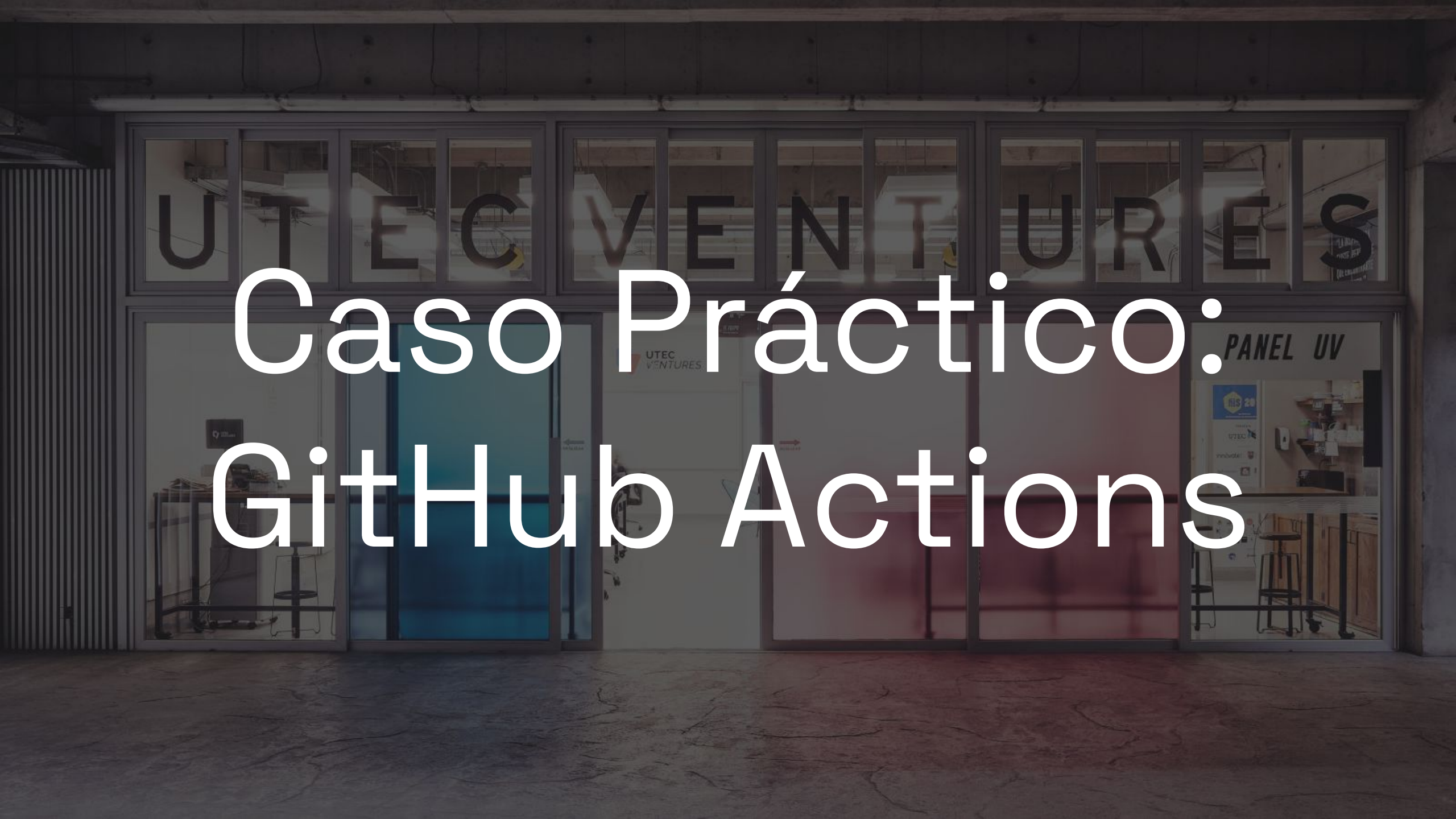


A background image showing two women in an office. The woman on the left is smiling broadly, looking towards the right. The woman on the right is also smiling, looking at a computer screen and pointing at it with her right hand. The image is dimmed to serve as a background for the text.

Buenas Prácticas en DataOps

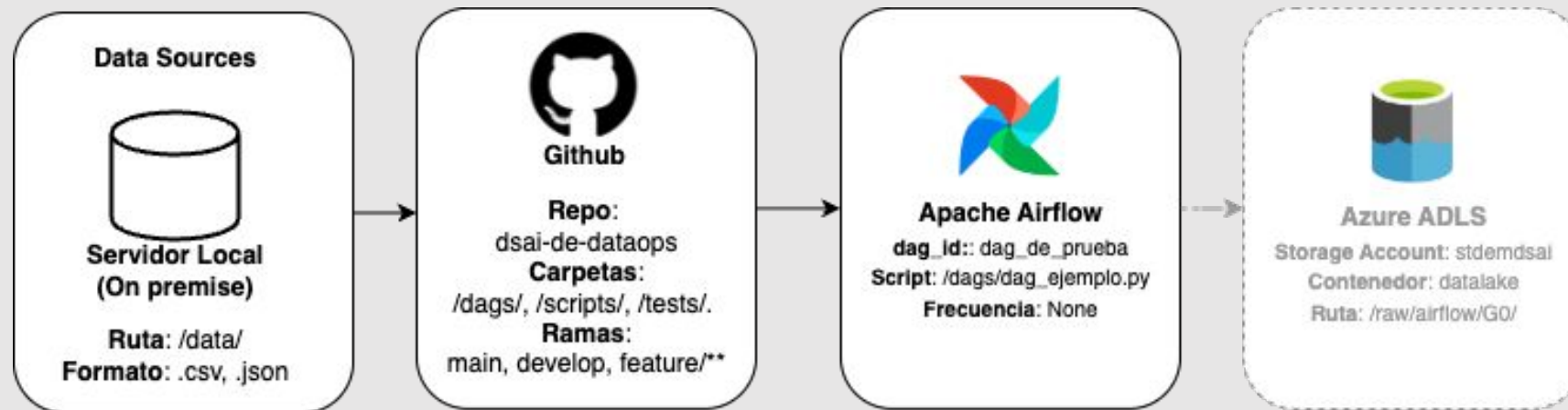
Buenas Prácticas en DataOps

Práctica	¿Por qué importa?	Airflow	Databricks
Git	Versionamiento	DAGs	Notebooks / Bundles
CI/CD	Automatización	GitHub Actions	GitHub Actions
Tests	Calidad	Pytest	Pytest
Observabilidad	Monitoreo	Logs + UI	Job Runs
IaC / DaC	Reproducibilidad	Terraform	Databricks Asset Bundles

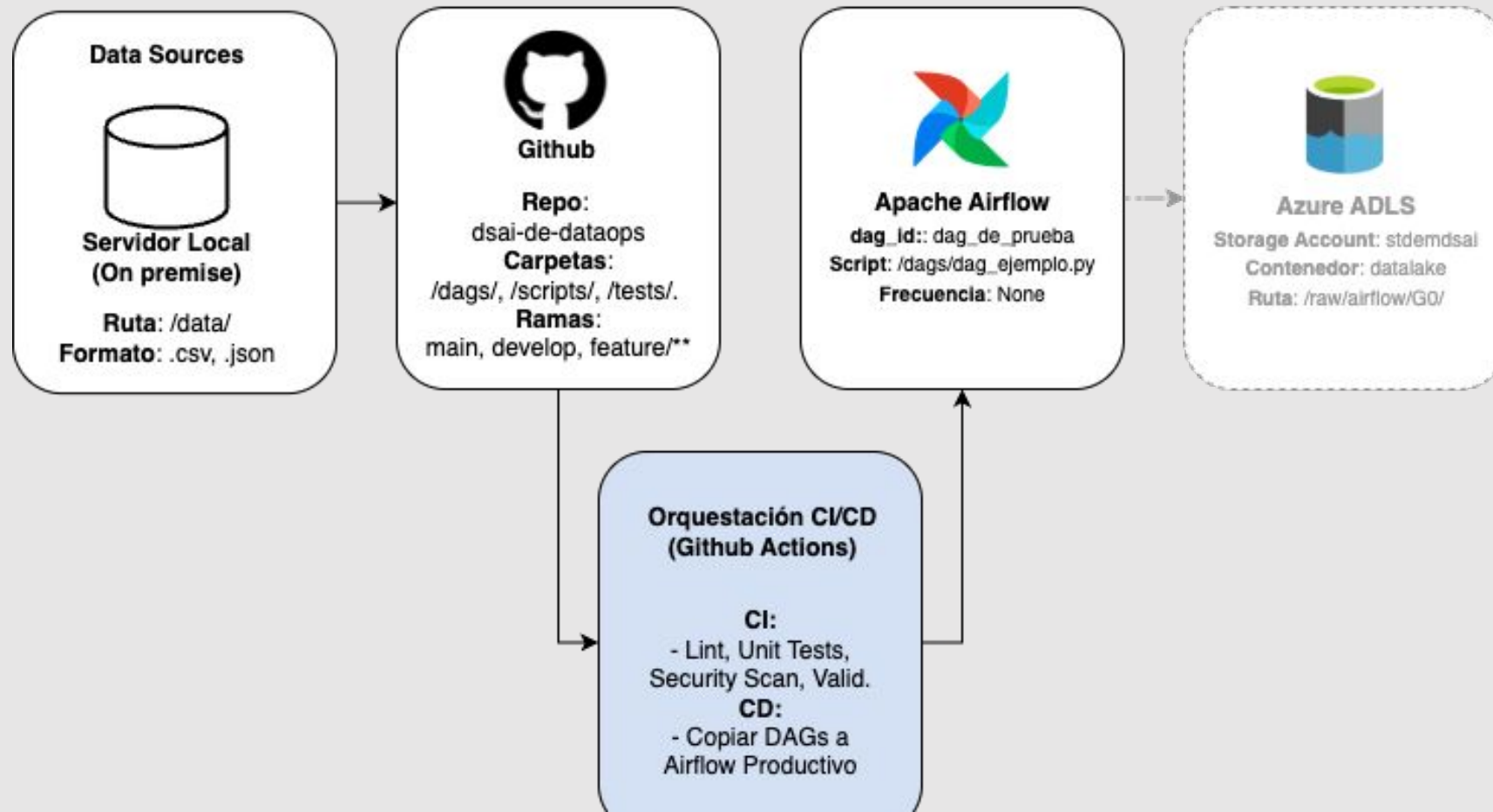


Caso Práctico: GitHub Actions

Arquitectura tradicional (sin DataOps)



Arquitectura con DataOps aplicado



¿Qué incluye un proceso DataOps completo?

- Diseño colaborativo del flujo de datos.
- Versionamiento del código (Git).
- Pruebas automáticas (CI): lógica + calidad de datos.
- Despliegue automático (CD): Airflow, Databricks.
- Observabilidad: monitoreo + logs + alertas.
- Feedback rápido: iteraciones ágiles.



Pasos para habilitar CI/CD con GitHub Actions

Escenario: Un repositorio con un DAG de Airflow en `/dags/etl_dag.py`.
Queremos:

- Desarrollar el código principal de proyectos
- Crear una carpeta `scripts/` para funciones reutilizables o validaciones
- Definir dependencias con `requirements.txt` y `requirements-dev.txt`
- Crear carpeta `tests/` con scripts de pruebas automáticas (pytest)
- Configurar el workflow de GitHub Actions en `.github/workflows/ci.yml`
- Ejecutar automáticamente validaciones, tests y linting en cada push o PR



Ejemplo práctico simple con GitHub Actions

Escenario: Un repositorio con un DAG de Airflow en `/dags/etl_dag.py`.
Queremos:

- Validar que no tenga errores de sintaxis.
- Desplegar (simulado) al hacer push a main.

Archivo `.github/workflows/dataops.yml`:



Ejemplo simple con GitHub Actions

Archivo `.github/workflows/dataops.yml`:

```
dataops.yml U X
.github > workflows > dataops.yml > {} jobs > {} validate-and-deploy > [ ] steps > {} 2
GitHub Workflow - YAML GitHub Workflow (github-workflow.json)
1  name: DataOps CI
2
3  on:
4    push:
5      branches: [main]
6
7  jobs:
8    validate-and-deploy:
9      runs-on: ubuntu-latest
10     steps:
11       - name: Clonar repositorio
12         uses: actions/checkout@v3
13
14       - name: Validar sintaxis del DAG
15         run: python -m py_compile dags/etl_dag.py
16
17       - name: Simular despliegue
18         run: echo "Desplegando DAG a Airflow..."
19
```



Ejemplo: Dataops - CI/CD

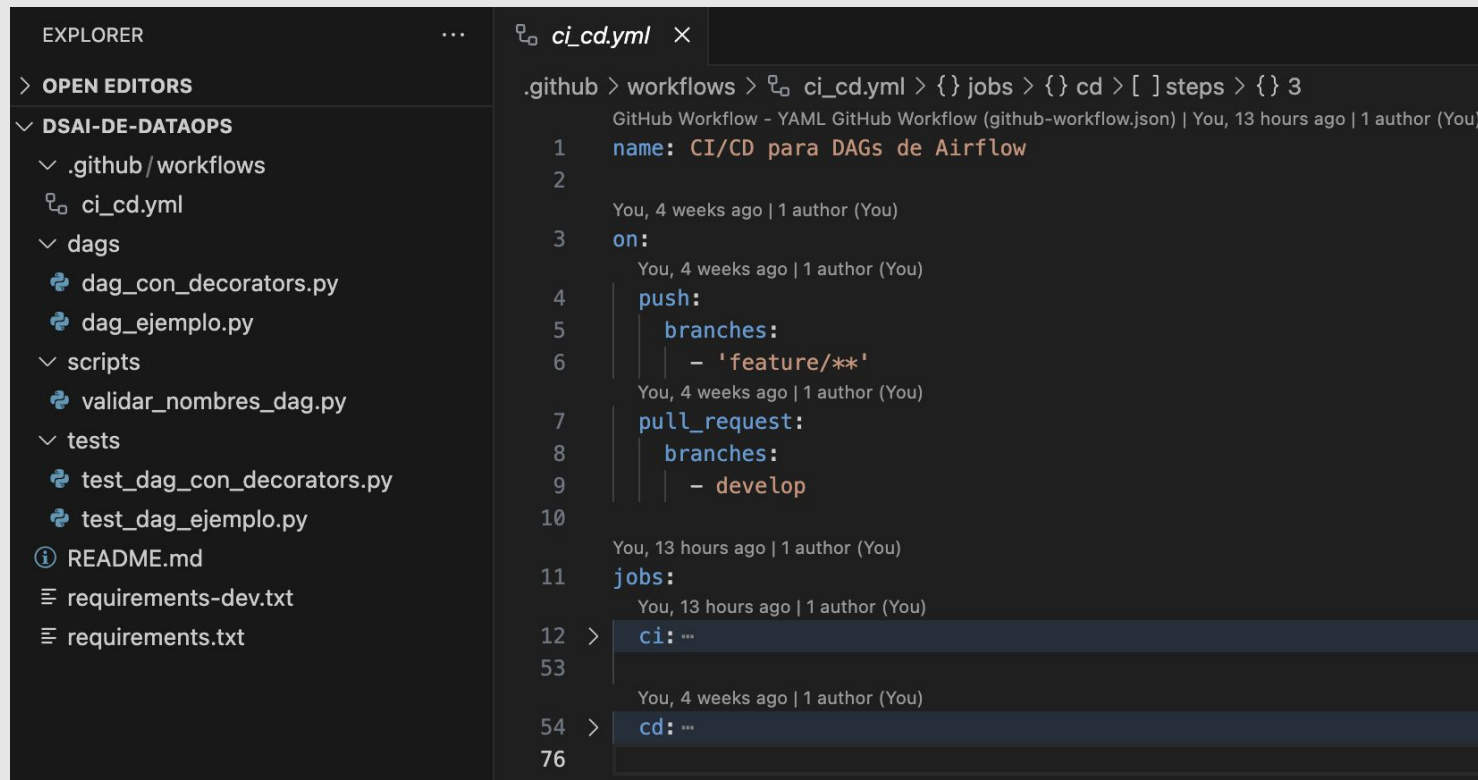
Escenario: Un repositorio con un DAG de Airflow en `/dags/etl_dag.py`.
Queremos:

- Validar que no tenga errores de sintaxis.
- Validar buenas prácticas de desarrollo.
- Ejecución de pruebas unitarias.
- Validar nomenclatura de DAGs.
- Escaneo de seguridad.
- Escaneo de dependencias.
- Desplegar (simulado) al hacer push a main.



Ejemplo: Dataops - CI/CD

Repositorio: <https://github.com/de-atintaya/dsai-de-dataops>



The screenshot shows a code editor interface with a sidebar on the left and a main editor area on the right. The sidebar, titled 'EXPLORER', shows a file tree for a repository named 'DSAI-DE-DATAOPS'. The tree includes folders like '.github/workflows', 'dags', 'scripts', and 'tests', along with files like 'ci_cd.yml', 'dag_con_decorators.py', 'dag_ejemplo.py', 'validar_nombres_dag.py', 'test_dag_con_decorators.py', 'test_dag_ejemplo.py', 'README.md', 'requirements-dev.txt', and 'requirements.txt'. The main editor area displays the content of 'ci_cd.yml', which is a GitHub Actions workflow. The workflow is named 'CI/CD para DAGs de Airflow' and is triggered on 'push' for 'feature/**' branches and 'pull_request' for 'develop' branches. The workflow consists of two jobs: 'ci' and 'cd'. The 'ci' job is currently selected and highlighted in blue. The editor shows line numbers from 1 to 76. The workflow file content is as follows:

```
1 name: CI/CD para DAGs de Airflow
2
3 on:
4   push:
5     branches:
6       - 'feature/**'
7   pull_request:
8     branches:
9       - develop
10
11 jobs:
12   ci: ...
13
14   cd: ...
```



The background image shows two women in an office. The woman on the left is smiling broadly, looking towards the right. The woman on the right is also smiling, looking at a computer screen and pointing at it with her right hand. The office environment is blurred in the background.

DataOps Herramientas

Herramientas comunes en DataOps

Categoría	Herramientas Comunes
Control de versiones	Git, GitHub, GitLab, Bitbucket
Integración Continua (CI)	GitHub Actions, TeamCity, GitLab CI, Jenkins
Despliegue Continuo (CD)	GitHub Actions, Octopus Deploy, ArgoCD, Azure DevOps
Infraestructura como código	Terraform, Pulumi, CloudFormation
Orquestación de flujo de datos	Apache Airflow, Prefect, Dagster



Takeaways

- DataOps extiende las prácticas de DevOps al mundo de los datos y los pipelines.
- CI/CD ayuda a detectar errores antes de que lleguen a producción.
- Las pruebas automáticas deben validar tanto código como calidad de datos.
- GitHub Actions puede automatizar validaciones, pruebas y despliegues de pipelines de datos.
- Terraform y Databricks Asset Bundles permiten gestionar entornos y recursos de forma reproducible
- La automatización reduce riesgos operativos y mejora la confiabilidad de los productos de datos.

¡Gracias!



UTEC Posgrado